

CS224N – Bài giảng 2: Word Vectors

Ghi chú chi tiết: Word2Vec, Negative Sampling, GloVe & Đánh giá Word Vectors

Ghi chú học tập – Natural Language Processing with Deep Learning

Stanford University

Mục lục

1	Biểu diễn Ý nghĩa của Từ (Word Meaning)	2
1.1	WordNet và các hạn chế	2
1.2	Từ như các ký hiệu rời rạc: Vector One-hot	2
2	Distributional Semantics – Ngữ nghĩa Phân bố	2
3	Khung Word2Vec	3
3.1	Giả định “Bag of Words” và hệ quả	3
3.2	Hai biến thể: CBOW và Skip-gram	3
3.3	Vì sao Skip-gram thường được xem là đơn giản hơn CBOW?	4
4	Hàm Mục tiêu & Tối ưu hóa trong Word2Vec	4
4.1	Hàm likelihood và hàm mất mát	4
4.2	Naive Softmax và nút thắt tính toán	4
4.3	Gradient Descent & Stochastic Gradient Descent (SGD)	5
4.4	Skip-gram với Negative Sampling (SGNS)	5
4.5	Phân phối Unigram lũy thừa $3/4$	6
5	Phương pháp dựa trên Đếm (Count-based) vs. Dự đoán trực tiếp	7
5.1	Ma trận đồng xuất hiện (Co-occurrence Matrix) và SVD	7
5.2	COALS	7
6	GloVe – Global Vectors for Word Representation	8
6.1	Trực giác cốt lõi: tỉ lệ xác suất đồng xuất hiện	8
6.2	Hàm mục tiêu của GloVe	8
7	Đánh giá Word Vectors	9
7.1	Intrinsic: Word Vector Analogies	9
7.2	Intrinsic: tương quan độ tương đồng (WordSim-353)	9
7.3	Extrinsic: Named Entity Recognition (NER)	9
8	Đa nghĩa (Polysemy) và Nghĩa của Từ (Word Senses)	9
9	Giới thiệu Neural Classifiers	10
9.1	Linear vs. Neural Classifiers	10
9.2	Window-based Neural Classifier	10
9.3	Cross-Entropy Loss	10

1 Biểu diễn Ý nghĩa của Từ (Word Meaning)

1.1 WordNet và các hạn chế

Trong NLP truyền thống, giải pháp phổ biến nhất để biểu diễn ý nghĩa là dùng **WordNet** – một *thesaurus* (từ điển đồng nghĩa) chứa danh sách các tập từ đồng nghĩa (*synonym sets*) và các quan hệ *hypernym* (quan hệ “là một loại của”, *is-a*). Ví dụ, ta có thể tra các synset của từ “good” hay chuỗi hypernym của “panda” (panda → carnivore → mammal → ... → entity).

Tuy hữu ích, WordNet có nhiều **hạn chế** cốt lõi:

- **Thiếu sắc thái ngữ nghĩa (nuance):** ví dụ “proficient” được liệt kê là đồng nghĩa của “good”, nhưng điều này chỉ đúng trong một số ngữ cảnh.
- **Thiếu từ mới / slang:** các từ như *wicked*, *badass*, *nifty*, *wizard*, *ninja* liên tục xuất hiện; việc cập nhật thủ công là bất khả thi.
- **Mang tính chủ quan và đòi hỏi lao động thủ công** lớn để xây dựng và bảo trì.
- **Không tính được độ tương đồng (similarity)** giữa các từ một cách chính xác – đây là hạn chế nghiêm trọng nhất đối với các mô hình học máy.

1.2 Từ như các ký hiệu rời rạc: Vector One-hot

Trong NLP cổ điển, mỗi từ được xem là một **ký hiệu rời rạc** (a *localist representation*), biểu diễn bằng một **vector one-hot**: một vector có đúng một thành phần bằng 1, còn lại bằng 0. Số chiều của vector bằng kích thước từ vựng $|V|$ (thường 500,000+):

$$e_{\text{motel}} = [0 \ 0 \ \dots \ 0 \ 1 \ 0 \ \dots \ 0]^\top, \quad e_{\text{hotel}} = [0 \ \dots \ 0 \ 1 \ 0 \ \dots \ 0]^\top, \quad e_w \in \mathbb{R}^{|V|}. \quad (1)$$

Vấn đề chính: tính trực giao (orthogonality). Xét bài toán tìm kiếm web: khi người dùng gõ “Seattle motel”, ta muốn khớp cả các tài liệu chứa “Seattle hotel”. Nhưng với biểu diễn one-hot, hai vector e_{motel} và e_{hotel} **trực giao** với nhau:

$$e_{\text{motel}}^\top e_{\text{hotel}} = 0. \quad (2)$$

Tích vô hướng bằng 0 nghĩa là **không có khái niệm tương đồng tự nhiên** nào giữa các từ. Mọi cặp từ đều “cách đều” nhau, dù “motel” và “hotel” rõ ràng gần nghĩa. Dựa vào danh sách đồng nghĩa của WordNet để vá lỗi này thất bại vì tính không đầy đủ. **Giải pháp:** thay vì áp đặt từ bên ngoài, hãy *học* cách mã hóa độ tương đồng *ngay trong bản thân các vector*.

2 Distributional Semantics – Ngữ nghĩa Phân bố

Ý tưởng nền tảng của toàn bộ bài giảng đến từ *ngữ nghĩa phân bố*:

“You shall know a word by the company it keeps.”

— J. R. Firth (1957)

Distributional semantics: ý nghĩa của một từ được quyết định bởi những từ thường xuyên xuất hiện gần nó. Đây là một trong những ý tưởng thành công nhất của NLP thống kê hiện đại.

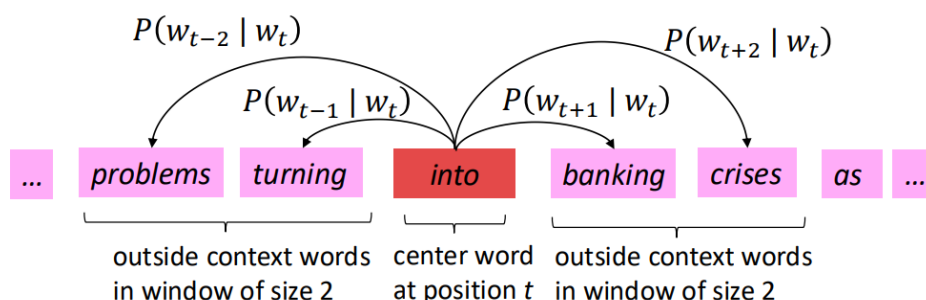
Khi một từ w xuất hiện trong văn bản, **ngữ cảnh (context)** của nó là tập các từ nằm trong một *cửa sổ* có kích thước cố định bao quanh w . Bằng cách tổng hợp *rất nhiều* ngữ cảnh của w trên toàn bộ corpus, ta xây dựng được một biểu diễn cho w . Ví dụ, mọi lần từ “banking” xuất hiện, các từ như *debt*, *crises*, *regulation*, *system* xung quanh nó sẽ dần dần “định nghĩa” nên “banking”.

Ta xây dựng một **dense vector** (vector dày đặc) cho mỗi từ, sao cho vector của các từ xuất hiện trong ngữ cảnh tương tự sẽ gần nhau; độ tương đồng được đo bằng **tích vô hướng (dot product)**. Các vector này còn được gọi là *word embeddings* hay *neural word representations*, và chúng là một *distributed representation* (biểu diễn phân tán – thông tin trải trên nhiều chiều thay vì tập trung ở một chiều như one-hot).

3 Khung Word2Vec

Word2Vec (Mikolov et al., 2013) là một khung (framework) để *học* word vectors. Ý tưởng tổng quát:

- Có một **corpus** lớn (“body” of text) – một danh sách dài các từ.
- Mỗi từ trong từ vựng cố định được biểu diễn bằng một **vector**.
- Duyệt qua từng vị trí t trong văn bản; tại đó có một **từ trung tâm** c (center word) và các **từ ngữ cảnh** o (outside/context words).
- Dùng *độ tương đồng* giữa các vector của c và o để tính *xác suất* $P(o | c)$ (hoặc ngược lại).
- **Điều chỉnh liên tục** các vector để *tối đa hóa* xác suất này.



Hình 1: Enter Caption

Cửa sổ kích thước $m = 2$: từ trung tâm “banking” dự đoán các từ ngữ cảnh xung quanh (mô hình Skip-gram).

3.1 Giả định “Bag of Words” và hệ quả

Word2Vec là một mô hình **“Bag of Words”**: nó *bỏ qua thứ tự và khoảng cách chính xác* của các từ bên trong cửa sổ ngữ cảnh. Mô hình đưa ra *cùng một phân phối dự đoán* cho mọi vị trí ngữ cảnh, bất kể từ đó nằm bên trái hay bên phải, gần hay xa từ trung tâm.

Điều này thoát nghe là một *nhược điểm* (mất thông tin vị trí), nhưng lại chính là mục tiêu mong muốn: ta muốn một mô hình gán *xác suất tương đối cao* cho *mọi* từ thực sự xuất hiện trong ngữ cảnh (đủ thường xuyên), thay vì mô hình hóa vị trí chính xác. Việc đơn giản hóa này giúp học được các biểu diễn ngữ nghĩa mạnh mẽ.

3.2 Hai biến thể: CBOW và Skip-gram

- **Skip-gram (SG)**: Dự đoán các *từ ngữ cảnh* (“outside”) – độc lập theo vị trí – *cho trước* từ trung tâm. (Đây là biến thể được trình bày chính trong khóa học.)
- **Continuous Bag of Words (CBOW)**: Dự đoán *từ trung tâm* từ (túi) các từ ngữ cảnh.

3.3 Vì sao Skip-gram thường được xem là đơn giản hơn CBOW?

Đây là một điểm mấu chốt về mặt cài đặt và toán học:

- **Skip-gram tách bài toán thành các dự đoán độc lập.** Với một từ trung tâm c duy nhất, mỗi từ ngữ cảnh o trong cửa sổ được dự đoán *riêng biệt* qua một số hạng $P(o | c)$. Hàm likelihood *phân rã* (*factorizes*) thành tích của các số hạng độc lập, mỗi cặp (center, context) là một mẫu huấn luyện riêng. Chỉ có *một* vector đầu vào $\mathbf{v}_c$ tham gia mỗi lần dự đoán.
- **CBOW phải gộp (aggregate) nhiều vector ngữ cảnh lại.** Để dự đoán từ trung tâm, CBOW cần *cộng/lấy trung bình* nhiều vector ngữ cảnh thành một vector duy nhất trước khi tính xác suất. Việc gộp này *ràng buộc các từ ngữ cảnh với nhau*, khiến cấu trúc gradient phức tạp hơn (gradient của lỗi phải được phân bổ ngược lại cho tất cả các vector ngữ cảnh trong tuple).

Nói cách khác: Skip-gram xử lý mỗi vị trí ngữ cảnh một cách *độc lập* (1 center $\rightarrow$ nhiều dự đoán 1-1), cho gradient sạch và trực tiếp; còn CBOW trộn n vector ngữ cảnh $\rightarrow$ 1 dự đoán, làm phép toán rườm rà hơn. Vì thế Skip-gram được xem là *đơn giản, dễ cài đặt hơn*, và cũng thường hoạt động tốt hơn với từ hiếm.

4 Hàm Mục tiêu & Tối ưu hóa trong Word2Vec

4.1 Hàm likelihood và hàm mất mát

Với mỗi vị trí $t = 1, \dots, T$, ta dự đoán các từ ngữ cảnh trong cửa sổ cố định kích thước m , cho trước từ trung tâm w_t . **Data likelihood**:

$$L(\theta) = \prod_{t=1}^T \prod_{\substack{-m \leq j \leq m \\ j \neq 0}} P(w_{t+j} | w_t; \theta), \quad (3)$$

trong đó θ là *tất cả* các biến (vector) cần tối ưu. Hàm mục tiêu $J(\theta)$ là **average negative log-likelihood** (đôi khi gọi là hàm *cost* hoặc *loss*):

$$J(\theta) = -\frac{1}{T} \log L(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log P(w_{t+j} | w_t; \theta). \quad (4)$$

Tối thiểu hóa $J(\theta) \iff$ tối đa hóa độ chính xác dự đoán.

4.2 Naive Softmax và nút thắt tính toán

Để tính $P(o | c)$, ta dùng **hai vector cho mỗi từ w** :

- $\mathbf{v}_w$: khi w là từ trung tâm (center word);
- $\mathbf{u}_w$: khi w là từ ngữ cảnh (context/outside word).

Khi đó, với từ trung tâm c và từ ngữ cảnh o , ta dùng hàm **softmax**:

$$P(o | c) = \frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{w \in V} \exp(\mathbf{u}_w^\top \mathbf{v}_c)}. \quad (5)$$

Diễn giải công thức (5) theo ba bước:

1. **Dot product** $\mathbf{u}_o^\top \mathbf{v}_c$ đo *độ tương đồng* giữa o và c : tích vô hướng càng lớn $\Rightarrow$ xác suất càng cao.
2. **Mũ hóa** $\exp(\cdot)$ biến mọi giá trị thành số dương.
3. **Chuẩn hóa** bằng mẫu số (tổng trên toàn từ vựng V) để thu được một phân phối xác suất hợp lệ.

Nút thắt tính toán. Mẫu số của (5) là *một tổng khổng lồ trên toàn bộ từ vựng* $\sum_{w \in V}$. Với từ vựng lớn ($|V|$ hàng trăm nghìn tới hàng triệu), việc tính mẫu số này cho *mỗi* bước là cực kỳ tốn kém – chi phí $O(|V|)$ mỗi lần cập nhật. Đây chính là động lực dẫn tới Negative Sampling.

4.3 Gradient Descent & Stochastic Gradient Descent (SGD)

Để huấn luyện, ta “đi xuống dốc” theo hướng gradient âm. Quy tắc cập nhật:

$$\theta^{\text{new}} = \theta^{\text{old}} - \alpha \nabla_{\theta} J(\theta), \quad \theta_j^{\text{new}} = \theta_j^{\text{old}} - \alpha \frac{\partial}{\partial \theta_j^{\text{old}}} J(\theta), \quad (6)$$

trong đó α là **learning rate** (step size), thường rất nhỏ để tránh “vượt quá” cực tiểu. Nhắc lại rằng $\theta \in \mathbb{R}^{2dV}$ gom *tất cả* tham số: với vector d chiều và V từ, mỗi từ có *hai* vector nên tổng cộng có $2dV$ tham số.

Vấn đề của Gradient Descent đầy đủ. $J(\theta)$ là hàm của *tất cả* các cửa sổ trong corpus (có thể hàng tỉ!), nên $\nabla_{\theta} J(\theta)$ cực kỳ đắt để tính – ta phải chờ rất lâu chỉ để thực hiện *một* lần cập nhật. Đây là ý tưởng tồi cho hầu hết mọi mạng nơ-ron.

Giải pháp: SGD. Stochastic Gradient Descent lặp lại việc lấy mẫu một cửa sổ (hoặc một *mini-batch*, ví dụ 16 hoặc 32 mẫu) và cập nhật tham số sau mỗi lần. SGD nhanh hơn nhiều bậc và nhiễu ngẫu nhiên của nó còn giúp mô hình “thoát” khỏi các cực tiểu địa phương xấu hoặc điểm yên ngựa. Trong mỗi cửa sổ ta chỉ có nhiều nhất $2m + 1$ từ (cộng $2km$ từ âm với negative sampling), nên $\nabla_{\theta} J_t(\theta)$ *rất thưa (sparse)* – ta chỉ cần cập nhật các hàng tương ứng của ma trận embedding U và V .

4.4 Skip-gram với Negative Sampling (SGNS)

Thay vì tính softmax đầy đủ (đắt vì mẫu số), Word2Vec chuẩn trong thực tế cài đặt Skip-gram với **Negative Sampling**. *Ý tưởng chính*: huấn luyện các **binary logistic regression** để *phân biệt*:

- một **cặp thật** (từ trung tâm và một từ thật trong cửa sổ ngữ cảnh), với một số **cặp “nhiều”** (từ trung tâm ghép với một từ ngẫu nhiên).

Ta *tối đa hóa* xác suất từ ngữ cảnh thật xuất hiện, và *tối thiểu hóa* xác suất các từ ngẫu nhiên xuất hiện quanh từ trung tâm. Hàm mất mát cho một cặp (theo ký hiệu nhất quán của khóa học):

$$J_{\text{neg-sample}}(\mathbf{u}_o, \mathbf{v}_c, U) = -\log \sigma(\mathbf{u}_o^\top \mathbf{v}_c) - \sum_{k \in \{K \text{ sampled indices}\}} \log \sigma(-\mathbf{u}_k^\top \mathbf{v}_c), \quad (7)$$

với hàm **logistic/sigmoid**

$$\sigma(x) = \frac{1}{1 + e^{-x}} \in (0, 1). \quad (8)$$

Định nghĩa nghiêm ngặt từng biến trong (7).

- $\mathbf{v}_c$ – **vector của từ trung tâm** (center word) c ; là vector “đầu vào” mà ta đang xét tại vị trí hiện tại.
- $\mathbf{u}_o$ – **vector của từ ngữ cảnh THẬT** (true outside word) o , tức từ thực sự xuất hiện trong cửa sổ quanh c . Số hạng $-\log \sigma(\mathbf{u}_o^\top \mathbf{v}_c)$ *đẩy* $\mathbf{u}_o$ và $\mathbf{v}_c$ lại gần nhau (tăng dot product).
- $\mathbf{u}_k$ – **vector của từ âm (negative sample) thứ k** : một từ được *lấy mẫu ngẫu nhiên* từ từ vựng, đóng vai trò “nhiều”. Dấu trừ trong $\sigma(-\mathbf{u}_k^\top \mathbf{v}_c)$ khiến số hạng này *đẩy* $\mathbf{u}_k$ ra xa $\mathbf{v}_c$ (giảm dot product).
- K – **số lượng mẫu âm** lấy cho mỗi cặp thật (thường $K \approx 5-20$); $\{K \text{ sampled indices}\}$ là tập chỉ số của các từ âm đó.
- U – ma trận (tập hợp) các vector ngữ cảnh; σ – hàm sigmoid thay cho softmax (do đó không còn tổng trên toàn V).

Vì sao SGNS hiệu quả hơn? Softmax đầy đủ cần tính tổng trên toàn bộ $|V|$ ở mẫu số cho mỗi bước. SGNS thay tổng đó bằng chỉ K phép so sánh nhị phân, biến bài toán phân loại $|V|$ -lớp thành $K + 1$ bài toán phân loại nhị phân *nhỏ*. Chi phí giảm từ $O(|V|)$ xuống $O(K)$ với $K \ll |V|$.

4.5 Phân phối Unigram lũy thừa $3/4$

Các mẫu âm không được lấy đều (uniform) cũng không theo tần suất thô, mà theo phân phối **unigram lũy thừa $3/4$** :

$$P_n(w) = \frac{U(w)^{3/4}}{Z}, \quad Z = \sum_{w' \in V} U(w')^{3/4}, \quad (9)$$

trong đó $U(w)$ là phân phối unigram (tần suất) của từ w , và Z là hằng số chuẩn hóa.

Mục đích toán học. Hàm $x \mapsto x^{3/4}$ là hàm *lõm* (concave) trên $[0, 1]$, nên nó *nâng* các giá trị nhỏ lên tương đối nhiều hơn các giá trị lớn. Cụ thể, sau khi chuẩn hóa theo (9):

- Các từ **hiếm** (tần suất thấp) được *tăng* xác suất được lấy mẫu so với tần suất thô của chúng.
- Các từ **cực phổ biến** (như *the, a, of*) bị *giảm* xác suất tương đối, tránh chi phối toàn bộ các mẫu âm.

Vì sao tốt hơn unigram thô hay uniform (thực nghiệm)? Số mũ $3/4$ nằm *giữa* hai thái cực:

- Số mũ = 1 (unigram thô): các từ chức năng siêu phổ biến bị lấy mẫu quá nhiều, làm mẫu âm kém thông tin.
- Số mũ = 0 (uniform): mọi từ như nhau, bỏ qua hoàn toàn thông tin tần suất, khiến từ hiếm bị lấy mẫu quá mức.

Giá trị $3/4$ “làm phẳng” phân phối vừa đủ: từ hiếm được lấy mẫu thường xuyên hơn, từ phổ biến vẫn được ưu tiên nhưng không áp đảo – một sự cân bằng được chứng minh là tốt nhất về mặt *thực nghiệm*.

5 Phương pháp dựa trên Đếm (Count-based) vs. Dự đoán trực tiếp

Có một điều “kỳ lạ” khi lặp qua toàn bộ corpus nhiều lần: tại sao không *trực tiếp tích lũy* thống kê về việc từ nào xuất hiện gần từ nào?

5.1 Ma trận đồng xuất hiện (Co-occurrence Matrix) và SVD

Ta xây dựng **ma trận đồng xuất hiện** X với hai lựa chọn:

- **Window-based** (giống Word2Vec): dùng cửa sổ quanh mỗi từ, nắm bắt cả thông tin cú pháp lẫn ngữ nghĩa (“word space”).
- **Word-document**: đếm số lần từ xuất hiện trong mỗi tài liệu, cho ta các *chủ đề* chung (mọi thuật ngữ thể thao có entry giống nhau) – dẫn tới *Latent Semantic Analysis* (“document space”).

Vector đếm thô có *số chiều rất lớn* (tăng theo từ vựng) và *thưa*, gây các vấn đề về lưu trữ và độ bền của mô hình. Ta muốn các **vector số chiều thấp** (thường 25–1000 chiều), cô đọng “hầu hết” thông tin quan trọng.

Singular Value Decomposition (SVD). Ta phân rã ma trận đồng xuất hiện X thành:

$$X = U\Sigma V^T, \quad (10)$$

trong đó U và V là các ma trận *trực chuẩn* (*orthonormal*) – các cột là vector đơn vị và trực giao. Giữ lại chỉ k giá trị kỳ dị (singular values) lớn nhất cho ta $\hat{X}$, là **xấp xỉ hạng k tốt nhất** của X theo nghĩa bình phương tối thiểu (kết quả cổ điển Eckart–Young). Nhược điểm: *đắt* khi tính cho ma trận lớn.

Các “hacks” quan trọng. Chạy SVD trên *đếm thô* hoạt động rất tệ, vì các từ chức năng (*the, he, has*) quá phổ biến khiến cú pháp lẫn át. Một số cách khắc phục:

- Lấy *log* của tần suất: $\log(1 + X_{ij})$.
- Chặn trên: $\min(X, t)$ với $t \approx 100$.
- Bỏ qua các từ chức năng.
- *Ramped windows*: đếm từ gần nhiều hơn từ xa.
- Dùng *tương quan* thay cho đếm, rồi đặt giá trị âm về 0.

5.2 COALS

COALS (*Correlated Occurrence Analogue to Lexical Semantics*, Rohde et al., 2005) là một mô hình dựa trên đếm được cải tiến, gộp nhiều “hacks” trên lại một cách có nguyên tắc:

- Áp dụng **log-frequency transformation** để giảm ảnh hưởng của các từ quá phổ biến.
- Dùng **Pearson correlations** (tương quan Pearson) thay cho đếm thô, rồi cắt các giá trị âm về 0.
- Dùng *window ramping* (cửa sổ có trọng số theo khoảng cách).

Kết quả bất ngờ và quan trọng: các mô hình dựa trên đếm được chuẩn hóa tốt *cũng* nắm bắt được các **thành phần ngữ nghĩa tuyến tính** (**linear semantic components**) – ví dụ các cặp DRIVE → DRIVER, SWIM → SWIMMER, TEACH → TEACHER tạo thành các vector song song trong không gian.

6 GloVe – Global Vectors for Word Representation

GloVe (Pennington, Socher & Manning, EMNLP 2014) *kết hợp* ưu điểm của cả hai họ phương pháp:

- Ưu điểm *thống kê toàn cục* của phân rã ma trận (count-based): tận dụng toàn bộ thống kê đồng xuất hiện, huấn luyện nhanh, mở rộng được cho corpus khổng lồ.
- Ưu điểm *hiệu năng ngữ nghĩa* của các phương pháp của sở cục bộ (như Word2Vec): nắm bắt tốt các quan hệ tương tự và loại suy.

6.1 Trực giác cốt lõi: tỉ lệ xác suất đồng xuất hiện

Câu hỏi: Làm sao mã hóa *tỉ lệ (ratios)* của các xác suất đồng xuất hiện thành các *thành phần ngữ nghĩa tuyến tính* trong không gian vector?

Trả lời: dùng một mô hình **log-bilinear**. Ta muốn dot product của hai vector từ xấp xỉ log của xác suất đồng xuất hiện:

$$\mathbf{w}_i \cdot \mathbf{w}_j = \log P(i | j). \quad (11)$$

Khi đó, *hiệu* của các vector tương ứng với *tỉ lệ* các xác suất:

$$\mathbf{w}_x \cdot (\mathbf{w}_a - \mathbf{w}_b) = \log \frac{P(x | a)}{P(x | b)}. \quad (12)$$

Chính *tỉ lệ* (chứ không phải xác suất thô) mới mang thông tin phân biệt ngữ nghĩa: nếu x liên quan tới a hơn b thì tỉ lệ lớn, và ngược lại.

6.2 Hàm mục tiêu của GloVe

$$J = \sum_{i,j=1}^V f(X_{ij}) \left(\mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2, \quad (13)$$

với các thành phần:

- X_{ij} – số lần từ j xuất hiện trong ngữ cảnh của từ i .
- $\mathbf{w}_i, \tilde{\mathbf{w}}_j$ – vector từ trung tâm và vector từ ngữ cảnh (GloVe cũng dùng hai tập vector).
- $b_i, \tilde{b}_j$ – các *bias* vô hướng cho từng từ.
- $f(X_{ij})$ – **hàm trọng số**, giới hạn ảnh hưởng của các cặp đồng xuất hiện *quá thường xuyên* (bão hòa về 1 khi X_{ij} lớn) và không cho các cặp hiếm bị nhiễu chi phối:

$$f(x) = \begin{cases} (x/x_{\max})^\alpha, & x < x_{\max}, \\ 1, & x \geq x_{\max}, \end{cases} \quad \alpha = \frac{3}{4}, \quad x_{\max} \approx 100. \quad (14)$$

Ưu điểm: huấn luyện nhanh (*fast training*) và mở rộng được cho corpus rất lớn (*scalable to huge corpora*).

7 Đánh giá Word Vectors

Việc đánh giá liên quan tới phân biệt chung trong NLP: **Intrinsic** (nội tại) vs. **Extrinsic** (ngoại tại).

Loại	Đặc điểm	Ví dụ
Intrinsic	Đánh giá trên một <i>subtask</i> trung gian/cụ thể. Nhanh, giúp hiểu hệ thống, nhưng chưa chắc tương quan với tác vụ thực tế.	Word Analogies ($a : b :: c : ?$); tương quan với đánh giá của con người (WordSim-353).
Extrinsic	Đánh giá trên một <i>tác vụ thực</i> (downstream). Trực tiếp nhưng chậm, khó cô lập xem thành phần nào gây lỗi.	Named Entity Recognition (NER); Web Search / Information Retrieval.

7.1 Intrinsic: Word Vector Analogies

Ta đánh giá vector bằng cách xem *khoảng cách cosine* sau khi cộng vector có nắm bắt tốt các câu hỏi loại suy (analogy) ngữ nghĩa/cú pháp trực quan hay không, ví dụ “man : woman :: king : ?”. Về mặt hình thức, với câu hỏi $a:b :: c:d$, ta tìm:

$$d = \arg \max_i \frac{(x_b - x_a + x_c)^\top x_i}{\|x_b - x_a + x_c\|}. \quad (15)$$

Lưu ý: ta *loại bỏ* (*discard*) các từ đầu vào (a, b, c) khỏi phần tìm kiếm. **Vấn đề:** sẽ ra sao nếu thông tin có ở đó nhưng *không tuyến tính*?

7.2 Intrinsic: tương quan độ tương đồng (WordSim-353)

Ta so sánh khoảng cách vector với *đánh giá độ tương đồng của con người*. Ví dụ trong bộ dữ liệu WordSim-353, cặp (*tiger, cat*) được người gán 7.35/10, còn (*stock, jaguar*) chỉ 0.92. Một mô hình tốt sẽ có tương quan cao (ví dụ hệ số Spearman) giữa cosine similarity và điểm của con người. Trên các bộ như WS353, MC, RG, SCWS, RW, GloVe (huấn luyện trên 42B token) đạt tương quan cao nhất trong nhiều thiết lập.

7.3 Extrinsic: Named Entity Recognition (NER)

Một ví dụ điển hình nơi word vector tốt giúp *trực tiếp*: **NER** – nhận diện tham chiếu tới người, tổ chức, hay địa điểm, ví dụ “Chris Manning lives in Palo Alto” (Chris Manning → person, Palo Alto → location). Nếu thay đúng một hệ thống con bằng vector tốt hơn mà độ chính xác downstream tăng lên ⇒ thắng lợi rõ ràng.

8 Đa nghĩa (Polysemy) và Nghĩa của Từ (Word Senses)

Thách thức đa nghĩa. Nhiều từ mang *nhiều nghĩa phân biệt*. Ví dụ “*pike*” có thể là: một loài cá, một loại vũ khí, một con đường, hoặc một tư thế nhảy cầu; “*bank*” vừa là ngân hàng vừa là bờ sông. Một vector *duy nhất* cho mỗi từ dường như không đủ để tách các nghĩa này.

Giải pháp lịch sử: phân cụm nghĩa. Cách tiếp cận cũ là *gom cụm* (cluster) các lần sử dụng theo ngữ cảnh, tạo thành các vector nghĩa riêng biệt, ví dụ $bank_1, bank_2, \dots$. Mỗi cụm là một “nghĩa” và có vector riêng.

Quan điểm hiện đại: chồng chập tuyến tính (superposition). Một vector học được *duy nhất* thực chất là một **chồng chập tuyến tính có trọng số** (weighted linear superposition – tức trung bình có trọng số) của các vector nghĩa thành phần, với trọng số tỉ lệ theo *tần suất* của từng nghĩa trong corpus. Một cách xấp xỉ:

$$\mathbf{v}_{\text{word}} \approx \sum_s \frac{f_s}{\sum_{s'} f_{s'}} \mathbf{v}_{\text{sense}_s}, \quad (16)$$

với f_s là tần suất của nghĩa s .

Khôi phục nghĩa bằng Sparse Coding. Điều đáng chú ý: trong các không gian vector *số chiều cao*, các nghĩa được mã hóa đủ “thưa” (sparse) đến mức ta có thể *tái tạo lại* các nghĩa riêng lẻ từ một vector dày đặc gộp chung, bằng các kỹ thuật **sparse coding**. Nói cách khác, thông tin về từng nghĩa không bị mất mà chỉ bị “trộn” lại, và về mặt toán học vẫn có thể tách ra.

9 Giới thiệu Neural Classifiers

9.1 Linear vs. Neural Classifiers

- **Linear classifiers:** dùng *đầu vào cố định* với ma trận trọng số W được học, chỉ vẽ được các *ranh giới quyết định tuyến tính* (linear decision boundaries).
- **Neural classifiers:** *đồng thời* học cả biểu diễn đặc trưng phân tán (ví dụ chính các word vectors) lẫn trọng số các tầng. Nhờ đó tạo được *ranh giới quyết định phi tuyến* trên đầu vào thô, kết thúc bằng một tầng phân loại tuyến tính trên các đặc trưng ẩn trung gian.

9.2 Window-based Neural Classifier

Kiến trúc phân loại theo cửa sổ gồm ba tầng:

1. **Input layer:** *nối (concatenate)* các word vector trong một cửa sổ ngữ cảnh cố định thành một vector đầu vào x .
2. **Hidden layer:** chiếu x xuống không gian số chiều thấp hơn qua một biến đổi tuyến tính, rồi qua một hàm kích hoạt (activation function) f :

$$h = f(Wx + b). \quad (17)$$
3. **Output layer:** chấm điểm khả năng thuộc từng lớp bằng một phép chiếu tuyến tính ngoài, đưa qua sigmoid/softmax để ra xác suất các lớp.

9.3 Cross-Entropy Loss

Cho tác vụ phân loại, ta dùng **Cross-Entropy Loss**, đo độ lệch giữa phân phối mục tiêu thật p và phân phối dự đoán q :

$$H(p, q) = - \sum_x p(x) \log q(x). \quad (18)$$

Với nhãn mục tiêu dạng *1-hot* (chỉ một lớp đúng, $p(x) = 1$ tại lớp đó và 0 ở nơi khác), cross-entropy *rút gọn trực tiếp* thành **negative log-likelihood** của lớp đúng:

$$H(p, q) = - \log q(\text{lớp đúng}). \quad (19)$$

Key Goal. Hiểu rằng *ý nghĩa của từ có thể được biểu diễn bằng một vector số thực nhiều chiều*, và đủ nền tảng để *đọc được các bài báo về word embeddings* sau bài giảng này.